GROUP ASSIGNMENT REPORT

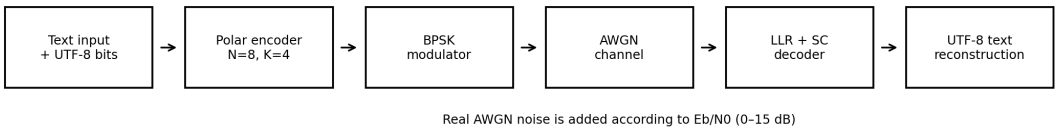
INFORMATION THEORY AND CODING (ITC)

Subject Code: EC24309

Polar Encoding and Decoding

over an AWGN Channel

End-to-end text transmission using BPSK and Successive Cancellation decoding



GROUP MEMBERS

S. No.	Name	Roll Number
1	Kshitiz Sinha	BTECH/10686/24
2	Himanshu Dubey	BTECH/10682/24
3	Pritika Mishra	BTECH/10671/24

Branch	Electronics and Communication Engineering (ECE)
Section	C
Subject	Information Theory and Coding (ITC)
Subject Code	EC24309
Submitted to	Dr. Sanjeet Kumar
Institution	Birla Institute of Technology, Mesra
Submission Date	4th October 2026

Contents

Abstract **1**

Objectives 1

1 Introduction **1**

2 Theoretical Background **2**

2.1 Polar transform and frozen bits 2

2.2 BPSK modulation 2

2.3 AWGN channel and noise variance 2

2.4 Channel log-likelihood ratio 3

2.5 Successive Cancellation decoding 3

3 Program Workflow **3**

3.1 Variable-length message support 4

4 Software Implementation **4**

5 Experimental Setup **5**

6 Simulation Results **6**

6.1 BER versus E_b/N_0 6

6.2 FER versus E_b/N_0 7

6.3 End-to-end message transmission tests 7

7 Discussion **8**

8 Software Validation **8**

9 Limitations and Future Improvements **9**

10 Conclusion **9**

References **9**

A Application Screenshots **10**

List of Figures

1 Functional block diagram of the simulated transmitter–channel–receiver chain. . . . 3

2 Measured Polar-coded BER and theoretical uncoded BPSK reference versus E_b/N_0. . . . 6

3 Measured Polar-coded FER versus E_b/N_0. . . . 7

4 Message transmission: original message, selected E_b/N_0, and recovered output. . . . 10

5 Block inspection: information bits, frozen-bit vector, codeword, BPSK symbols, AWGN samples, received values, LLRs, and SC decisions. . . . 10

6 BER versus E_b/N_0 graph from the running application. . . . 11

7 FER versus E_b/N_0 graph and the numerical results table. . . . 11

Abstract

This assignment implements and studies a short Polar-coded digital communication link over an additive white Gaussian noise (AWGN) channel. A user-entered UTF-8 message is converted into bits, framed with a 16-bit payload-length header, divided into 4-bit information blocks, and encoded using an $(N = 8, K = 4)$ Polar code. Each codeword is mapped to BPSK symbols and transmitted through an AWGN channel at E_b/N_0 values from 0 dB to 15 dB. At the receiver, channel log-likelihood ratios (LLRs) are calculated and a recursive Successive Cancellation (SC) decoder estimates the information bits. The decoded stream is reconstructed as UTF-8 text. A Monte Carlo experiment using 1,200 frames per SNR point measures bit error rate (BER) and frame error rate (FER). The measured errors decrease as E_b/N_0 increases; no errors were observed in the finite simulation runs from 7 dB to 15 dB.

Objectives

- Implement Polar encoding with frozen-bit insertion and the Arıkan transform over $GF(2)$.
- Transmit encoded bits using BPSK modulation through a simulated AWGN channel.
- Implement soft-input recursive Successive Cancellation decoding using the f and g LLR updates.
- Demonstrate variable-length UTF-8 message transmission and reconstruction.
- Evaluate BER and FER over E_b/N_0 values from 0 dB to 15 dB.

1 Introduction

Channel coding adds structured redundancy to transmitted data so that a receiver can detect and correct some errors caused by a noisy communication channel. Polar codes are a class of linear block codes based on channel polarization. They transform a group of input bits into a codeword while assigning selected input positions to information bits and fixing the remaining positions to known frozen values. The receiver uses the received channel observations to estimate the original information bits.

This project demonstrates the complete process in software. It is an educational, small-block implementation designed to make intermediate vectors, channel samples, and decoder decisions visible. The selected construction is not intended to represent the rate-matching and reliability-sequence procedures of a standards-compliant 5G New Radio (NR) Polar code.

2 Theoretical Background

2.1 Polar transform and frozen bits

The basic Polar kernel is the binary matrix

$$F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}. \tag{1}$$

For a block length $N = 2^m$, the generator matrix is the m-fold Kronecker product of F:

$$G_N = F^{\otimes m}, \qquad N = 2^m. \tag{2}$$

Encoding is performed over $GF(2)$, where addition is equivalent to XOR:

$$\mathbf{x} = \mathbf{u}G_N \pmod 2. \tag{3}$$

The input vector $\mathbf{u}$ contains N positions. In this assignment, $N = 8$ and $K = 4$. The four information bits are placed at zero-based positions $[3, 5, 6, 7]$; the frozen positions $[0, 1, 2, 4]$ are set to zero. The encoder applies the Polar transform to produce an 8-bit codeword.

Table 1: Polar code configuration used in the simulation.

Parameter	Value
Block length (N)	8
Information bits (K)	4
Frozen bits ($N - K$)	4
Code rate ($R = K/N$)	0.5
Information positions (zero-based)	$[3, 5, 6, 7]$
Frozen positions (zero-based)	$[0, 1, 2, 4]$

2.2 BPSK modulation

Each encoded bit is mapped to a real-valued BPSK symbol with unit energy:

$$0 \mapsto +1, \qquad 1 \mapsto -1. \tag{4}$$

2.3 AWGN channel and noise variance

The channel adds a zero-mean Gaussian random sample to each transmitted symbol. The received sample is

$$y = s + n, \qquad n \sim \mathcal{N}(0, \sigma^2), \tag{5}$$

where s is the transmitted symbol, n is independent Gaussian noise, and σ^2 is the noise variance. E_b/N_0 is expressed per information bit. With unit-energy BPSK and code rate

R, the real-valued noise variance used by the program is

$$\sigma^2 = \frac{1}{2R\,10^{(E_b/N_0)_{\mathrm{dB}}/10}}, \qquad R = \frac{K}{N}. \tag{6}$$

For this code, $R = 1/2$. Increasing E_b/N_0 reduces the noise variance, so the received samples tend to be closer to the transmitted BPSK symbols.

2.4 Channel log-likelihood ratio

The receiver converts each noisy BPSK observation into a log-likelihood ratio. Under the mapping $0 \mapsto +1$ and $1 \mapsto -1$, a positive LLR favours coded bit 0:

$$L(y) = \log \frac{p(y \mid x = 0)}{p(y \mid x = 1)} = \frac{2y}{\sigma^2}. \tag{7}$$

2.5 Successive Cancellation decoding

The recursive SC decoder uses soft information to estimate the input bits in their decoding order. The f operation combines LLRs to decode the first half of a constituent block. The g operation uses previously decoded partial sums to calculate LLRs for the second half. At frozen positions, the decoder forces the decision to zero; at information positions, the sign of the LLR determines the hard decision.

$$f(a, b) = 2\tanh^{-1}\left(\tanh\frac{a}{2}\ \tanh\frac{b}{2}\right), \tag{8}$$

$$g(a, b, \hat{u}) = b + (1 - 2\hat{u})a. \tag{9}$$

The implementation evaluates the f update using a numerically stable log-domain expression and evaluates the g update using the decoded partial sums from the left branch.

3 Program Workflow

The sequence shown in Figure 1 is repeated block-by-block: the framed bitstream is divided into 4-bit information blocks, and each block is independently encoded into an 8-bit codeword before modulation and transmission.

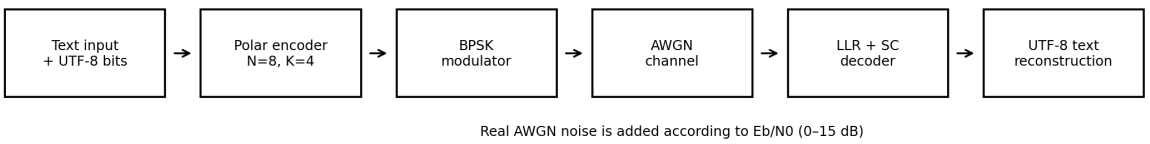


Figure 1: Functional block diagram of the simulated transmitter–channel–receiver chain.

1. **Message entry and UTF-8 conversion:** The user enters text. The program encodes it as UTF-8 bytes and converts each byte into eight bits.
2. **Framing:** A 16-bit header containing the payload length in bytes is prepended. Zero padding is appended, if required, so the bitstream length is divisible by $K = 4$.
3. **Block formation:** The framed stream is divided into 4-bit information blocks.
4. **Polar encoding:** For each block, four information bits are inserted at positions $[3, 5, 6, 7]$ in an 8-bit vector; the frozen positions are zero. The Polar transform creates an 8-bit codeword.
5. **BPSK and AWGN:** Code bits are mapped to $+1/-1$ symbols. Independent Gaussian noise is added using the selected E_b/N_0 value.
6. **Soft demodulation and decoding:** The receiver computes channel LLRs and applies the recursive SC decoder to each codeword.
7. **Message reconstruction:** Decoded information blocks are concatenated. The receiver reads the length header, removes padding, rebuilds the payload bytes, and decodes them as UTF-8 text.

3.1 Variable-length message support

The 4-bit information size is the size of each Polar input block, not the maximum length of the user message. A longer text is converted into a longer bitstream and split across as many 4-bit blocks as necessary. The 16-bit length header supports payloads up to 65,535 UTF-8 bytes. The message is reconstructed only if the relevant decoded bits, including the header, are recovered correctly.

4 Software Implementation

The project is implemented in Python. NumPy handles binary arrays, numerical operations, and random Gaussian samples. Matplotlib generates the performance plots. Streamlit provides the interactive user interface. Pytest is used for automated checks.

Table 2: Main project files and their responsibilities.

File	Purpose
`app.py`	Streamlit interface for message transmission, detailed block inspection, BER/FER simulation, and theory.
`polar_code.py`	Polar transform, information/frozen-bit placement, recursive SC decoding, and channel LLR calculation.
`channel.py`	BPSK modulation, E_b/N_0-to-noise-variance conversion, and AWGN generation.
`text_codec.py`	UTF-8 conversion, 16-bit payload-length header, padding, and message reconstruction.
`transmission.py`	Coordinates end-to-end message framing, encoding, transmission, decoding, and reconstruction.
`performance.py`	Monte Carlo simulation of BER and FER over the selected SNR values.
`test_core.py`	Automated checks for noiseless recovery of all 16 messages and encoder dimensions/bit placement.
`sionna_verify.py`	Optional reference-check script; it is not required to run the custom application.

5 Experimental Setup

Table 3: Simulation parameters.

Setting	Value
Polar block length (N)	8
Information bits per block (K)	4
Code rate	1/2
Modulation	BPSK
Channel	Real AWGN
Decoder	Recursive Successive Cancellation
E_b/N_0 sweep	0 dB to 15 dB, in 1 dB steps
Frames per SNR point	1,200
Information bits per SNR point	4,800
Performance experiment seed	Application default: 2026 (unless changed)

For each SNR point, 1,200 random 4-bit information frames were encoded and transmitted. Thus, 4,800 information bits were evaluated at every SNR point. A frame is counted as erroneous if at least one of its four decoded information bits differs from the transmitted information bits.

The error rates are calculated as

$$\mathrm{BER} = \frac{\text{number of erroneous information bits}}{4800}, \qquad \mathrm{FER} = \frac{\text{number of erroneous frames}}{1200}. \tag{10}$$

6 Simulation Results

The following table records the measured results from the Streamlit BER/FER experiment. BER is the number of erroneous information bits divided by 4,800. FER is the number of erroneous frames divided by 1,200.

Table 4: Measured BER and FER for the Polar-coded BPSK system.

E_b/N_0 (dB)	Bit errors	Frame errors	BER	FER
0	427	202	0.0889583	0.168333
1	305	135	0.0635417	0.112500
2	165	73	0.0343750	0.0608333
3	60	27	0.0125000	0.022500
4	28	14	0.00583333	0.0116667
5	6	3	0.00125000	0.002500
6	5	2	0.00104167	0.00166667
7	0	0	0	0
8	0	0	0	0
9	0	0	0	0
10	0	0	0	0
11	0	0	0	0
12	0	0	0	0
13	0	0	0	0
14	0	0	0	0
15	0	0	0	0

6.1 BER versus E_b/N_0

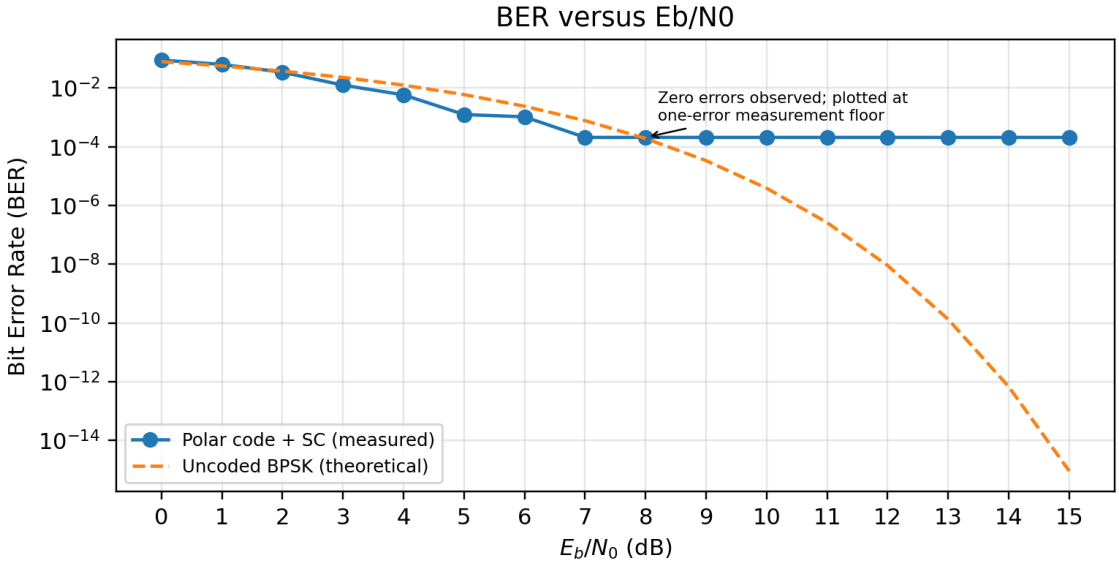


Figure 2: Measured Polar-coded BER and theoretical uncoded BPSK reference versus E_b/N_0.

The measured BER decreases from 0.0889583 at 0 dB to 0.00104167 at 6 dB. No information-bit errors were observed in the 1,200-frame runs at SNR values from 7 dB through 15 dB. The plot uses a one-error measurement floor (1/4800) for zero-count points because a logarithmic axis cannot display zero. The table retains the measured value of zero.

6.2 FER versus E_b/N_0

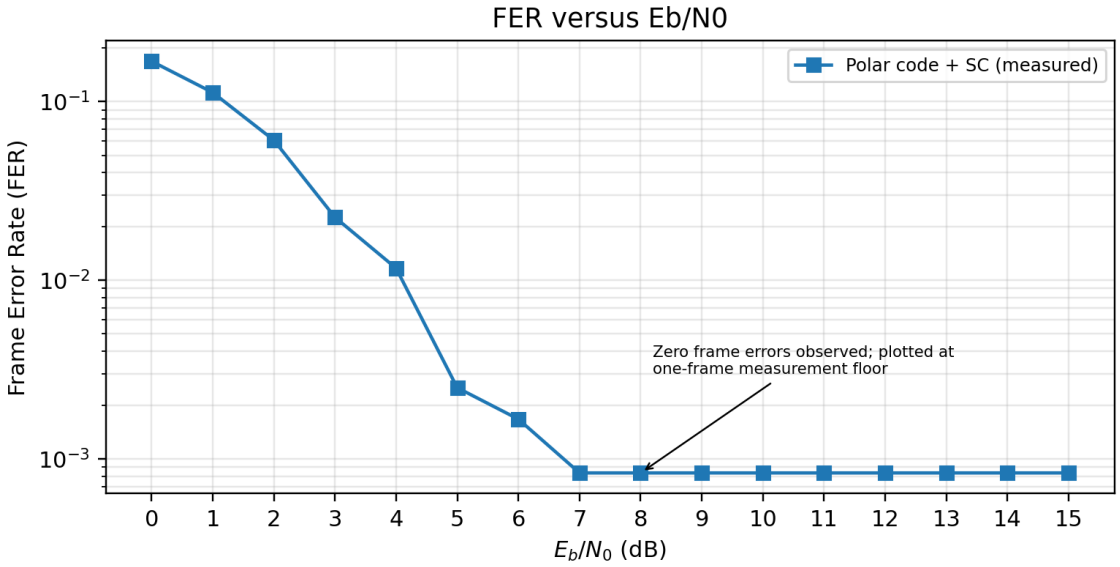


Figure 3: Measured Polar-coded FER versus E_b/N_0.

The measured FER decreases from 0.168333 at 0 dB to 0.00166667 at 6 dB. No erroneous frames were observed from 7 dB through 15 dB in the finite runs. The logarithmic plot places zero-count points at the one-frame measurement floor (1/1200); this is a plotting convention, not a measured nonzero error rate.

6.3 End-to-end message transmission tests

Four of the five demonstration messages were recovered exactly. The 0 dB test failed to reconstruct the original message. At low E_b/N_0, noise can cause information-bit errors or corrupt the decoded length header, preventing valid UTF-8 reconstruction. This demonstrates that message recovery depends on the quality of the noisy channel.

Table 5: Observed outcomes of the message transmission demonstration.

Test message	E_b/N_0	**Observed outcome**
HI	15 dB	Recovered successfully
HELLO WORLD	10 dB	Recovered successfully
English sentence	8 dB	Recovered successfully
Hindi UTF-8 message	15 dB	Recovered successfully
HELLO WORLD	0 dB	Exact recovery failed

7 Discussion

The overall BER and FER trends decrease as E_b/N_0 increases, which is consistent with the reduction in AWGN variance at higher E_b/N_0.

- At 0 dB, 427 information-bit errors and 202 erroneous frames were observed out of 4,800 bits and 1,200 frames, respectively.
- At 5 dB, the measured BER was 0.00125 and FER was 0.0025. At 6 dB, the measured BER was 0.00104167 and FER was 0.00166667.
- From 7 dB to 15 dB, the finite experiment observed zero bit errors and zero frame errors. This does not establish that the true BER or FER is exactly zero; more frames are needed to estimate rarer errors.
- The message demonstration complements the Monte Carlo experiment: it shows a user-entered payload passing through framing, block encoding, modulation, noise, decoding, and text reconstruction.

8 Software Validation

The project test suite was run using the command:

```
python -m pytest test_core.py -v
```

The observed test output was **2 passed in 0.32 seconds**. The project directory structure was also inspected and verified by the user.

Table 6: Automated test summary.

Automated test	Purpose	Observed result
`test_all_16_messages_decode_without_noise`	Checks that all $2^K = 16$ possible 4-bit information messages are recovered by the SC decoder when supplied with high-confidence, noiseless channel LLRs.	PASS
`test_encoder_returns_correct_lengths`	Checks that the encoder returns an $N = 8$ input vector and $N = 8$ codeword, and that information bits occupy the configured positions.	PASS

These tests verify core encoder/decoder properties; they are not an exhaustive proof of performance under every noise condition.

9 Limitations and Future Improvements

- The code uses a short $N = 8$, $K = 4$ educational construction with a fixed information set. It is not a 5G NR-compliant Polar code construction.
- The BER/FER experiment uses a finite sample of 1,200 frames per SNR point. More frames, especially at higher SNR, are needed to resolve very low error probabilities.
- The system does not include a CRC, retransmission protocol, channel interleaving, or practical synchronization and channel estimation.
- The current implementation uses one fixed block configuration. Larger Polar block lengths and a reliability-based information set could be implemented as an extension.
- An optional Sionna reference script is included, but the results in this report come from the custom NumPy implementation and do not claim an independent Sionna cross-check.

10 Conclusion

An end-to-end Polar-coded communication simulation over an AWGN channel was implemented in Python. The system accepts variable-length UTF-8 text, frames it with a payload-length header, encodes the bitstream in 4-bit information blocks using an $(8, 4)$ Polar code, performs BPSK modulation, adds Gaussian noise, and recovers the text using a recursive SC decoder. The Monte Carlo results show decreasing BER and FER as E_b/N_0 increases from 0 dB to 15 dB. In the 1,200-frame experiments, no errors were observed from 7 dB onwards, while the message tests showed that low-SNR noise can prevent exact reconstruction. The assignment therefore demonstrates both the encoding/decoding workflow and the effect of channel SNR on communication reliability.

References

[1] E. Arıkan, "Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Transactions on Information Theory*, vol. 55, no. 7, pp. 3051–3073, July 2009, doi: 10.1109/TIT.2009.2021379.

[2] Project implementation and experimental output: custom Python/NumPy Polar encoder, recursive SC decoder, BPSK/AWGN simulation, and Streamlit interface developed for this assignment.

A Application Screenshots

Insert screenshots captured directly from the running Streamlit application. The LaTeX source automatically includes a screenshot when a file with the indicated name is placed in the `figures/` folder. Otherwise, a labelled placeholder is shown.

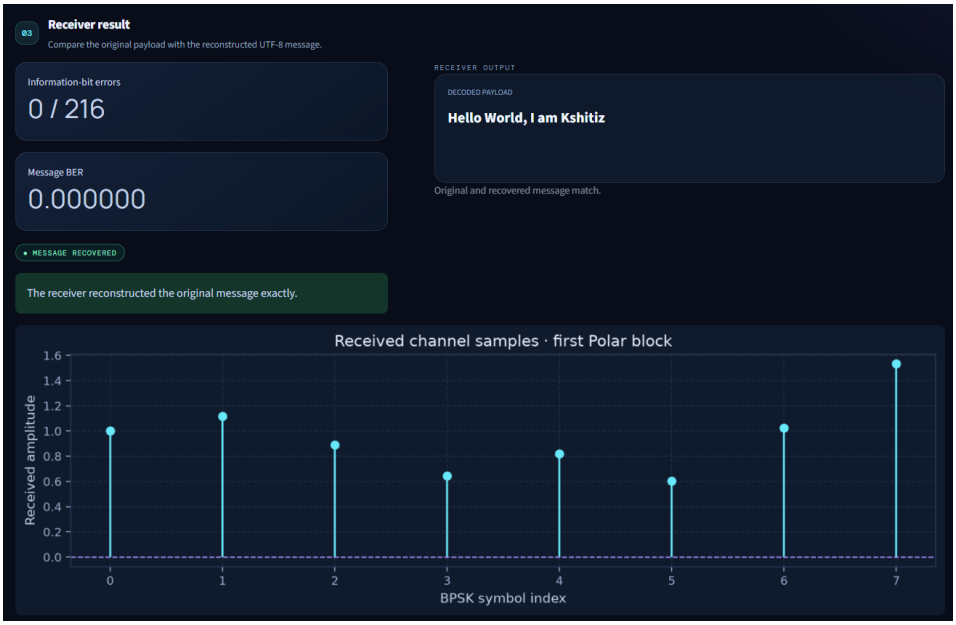


Figure 4: Message transmission: original message, selected E_b/N_0, and recovered output.

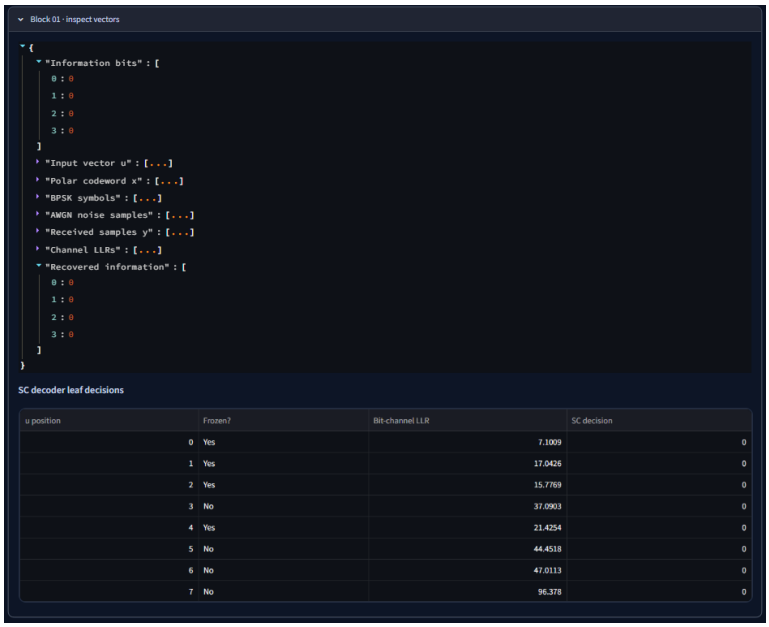


Figure 5: Block inspection: information bits, frozen-bit vector, codeword, BPSK symbols, AWGN samples, received values, LLRs, and SC decisions.

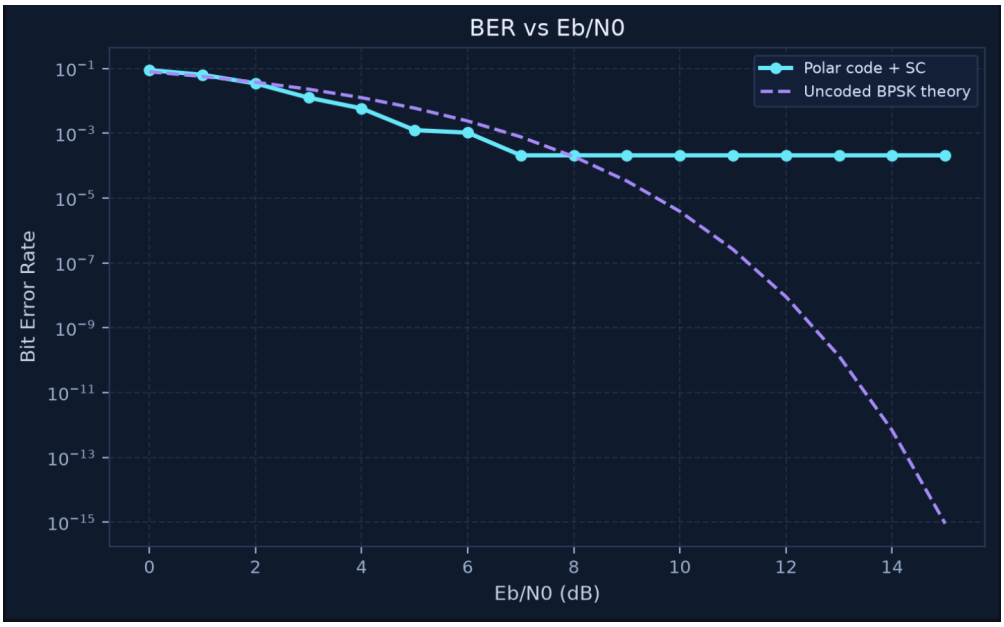


Figure 6: BER versus E_b/N_0 graph from the running application.

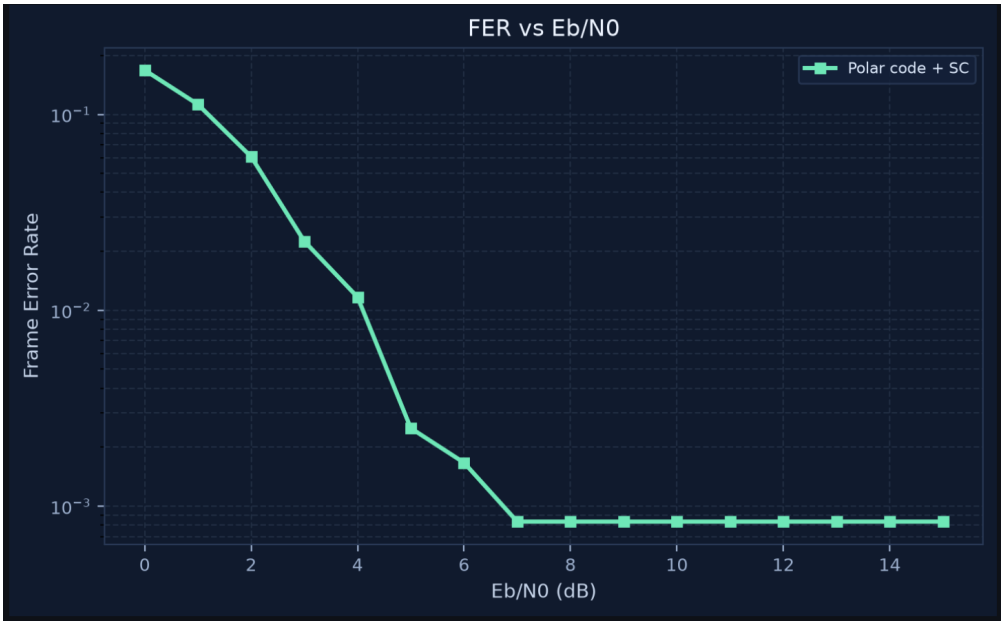


Figure 7: FER versus E_b/N_0 graph and the numerical results table.